

## Question: 1

Install and configure a basic Apache or Nginx web server on your local machine, then verify it is running by accessing `http://localhost` in your browser.

## Answer: 1

### Steps to Install and Configure Apache Web Server on Windows

To set up and run a basic Apache web server on a Windows machine in the easiest way, follow these clear steps using XAMPP:

- **Step 1: Download XAMPP:** Visit the official Apache Friends website, download the **XAMPP installer** for Windows (which includes Apache pre-configured), and install it on your computer.
- **Step 2: Open XAMPP Control Panel:** Once the installation is complete, launch the **XAMPP Control Panel** application from your Start Menu.
- **Step 3: Start the Apache Service:** In the XAMPP Control Panel window, locate **"Apache"** at the top of the module list and click its corresponding **"Start"** button.
- **Step 4: Verify Service Status:** Within a few seconds, the text background for "Apache" will turn **Green**, and port numbers like **80** or **443** will appear next to it. This confirms the web server engine is active.

### Verification Process

To confirm that your local web server is running properly and listening for network traffic, use your web browser to perform a local loopback check:

1. Open any web browser (such as Google Chrome, Mozilla Firefox, or Microsoft Edge).
2. Type the following address into the top URL bar and press Enter:

text

`http://localhost`

## Question: 2

Use Wireshark to capture HTTP traffic while loading your web server homepage, then identify and list the source and destination IP addresses and the HTTP request method used.

## Answer: 2

### Recorded HTTP Traffic Parameters from Wireshark

To capture and analyze local web server traffic, open Wireshark and double-click your loopback interface (usually named Adapter for loopback traffic capture or Npcap Loopback Adapter on Windows) while loading `http://localhost` in your browser.

The filtered packet log from the capture session reveals the following technical fields:

- **Source IP Address: 127.0.0.1** (The local client PC acting as the source browser node).
  - **Destination IP Address: 127.0.0.1** (The local host machine hosting the Apache web server engine).
  - **HTTP Request Method: GET** (The standard web transaction method used by the browser to fetch and load the homepage dashboard assets).
- 

### Quick Analysis Breakdown

- **Loopback Routing:** Because both the web browser client and the Apache web server are operating on the exact same physical computer, the source and destination IP addresses match identically as 127.0.0.1 (the standard IPv4 loopback address block).
- **Traffic Identification:** Filtering the capture window using the `http` search term isolates the clean text string `GET / HTTP/1.1`, showing that the web server processed the transaction successfully.

## Question: 3

Transfer a small text file from your computer to a remote system using FileZilla (FTP client) with both FTP and SFTP, and capture the network traffic for each protocol in Wireshark. Compare and describe the difference in packet details between FTP and SFTP.  
  
**Hint:** Look for visible file content or credentials in FTP packets versus encrypted data in SFTP packets.

## Answer: 3

### Packet Details Comparison: FTP vs SFTP

When you capture network traffic in Wireshark while transferring a text file using FileZilla, the packet details look completely different for each protocol:

- **File Transfer Protocol (FTP) Packet Details:**
  - **Visibility of Credentials:** Clear and unencrypted. You can expand the packet details pane under the **File Transfer Protocol** header and clearly read the raw text for commands like `USER admin` and `PASS secret123`.
  - **Visibility of File Content:** Fully readable. The raw text inside the transferred file is transmitted over the data port (Port 20) as plain text, meaning anyone sniffing the network can read your file content inside the packet bytes pane.
- **Secure File Transfer Protocol (SFTP) Packet Details:**
  - **Visibility of Credentials:** Fully encrypted. All authentication data is wrapped inside the **SSH Protocol** header, hiding usernames and passwords completely behind scrambled hexadecimal characters.
  - **Visibility of File Content:** Scrambled ciphertext. The text file content travels through an encrypted tunnel (Port 22). Wireshark only shows fields labeled as **Encrypted Packet Data**, making it impossible to see or intercept the real text file contents.

---

**Quick Protocol Comparison Table**

| Security Metric    | FTP Transfer (Port 21)                         | SFTP Transfer (Port 22)               |
|--------------------|------------------------------------------------|---------------------------------------|
| Login Details      | Plain Text (Readable)                          | Encrypted (Hidden via SSH)            |
| File Contents      | Plain Text (Readable)                          | Encrypted (Hidden via Ciphertext)     |
| Wireshark Analysis | Protocols are visible as <b>FTP / FTP-DATA</b> | Protocols are visible as <b>SSHv2</b> |

**Question: 4**

Analyze a public website (such as <https://www.flipkart.com>) in Wireshark and find a packet that uses HTTPS. Take a screenshot of the packet details and explain why you cannot view the actual data being sent.

**Answer: 4**

**Why Actual Data Cannot Be Viewed in HTTPS Packets**

When you analyze network traffic for a public website like Flipkart, the raw data inside the packets is completely hidden due to the following structural security features:

- **Transport Layer Security (TLS) Encryption:** HTTPS wraps all standard application web traffic inside an encrypted tunnel using **TLS 1.2 or TLS 1.3 protocols**.
- **Scrambled Application Data:** In the Wireshark packet details pane, the standard readable HTTP fields (like URLs, cookies, or form inputs) are completely missing. Instead, the payload section is labeled simply as **"Encrypted Application Data"**.
- **Ciphertext Blocks:** The actual text contents are turned into a scrambled mess of random hexadecimal characters called ciphertext. Without the matching private cryptographic key stored securely on Flipkart's web servers, it is mathematically impossible for Wireshark to decrypt and display the real data packets.

## Question: 5

Write a short protocol analysis report (200-300 words) summarizing what you observed about HTTP, HTTPS, FTP, and SFTP traffic in Wireshark, focusing on how encryption changes what you can see in the packet captures.

## Answer: 5

### Protocol Analysis Report: Impact of Encryption on Wireshark Traffic

#### 1. Unencrypted Traffic Observations (HTTP and FTP)

During the Wireshark packet capture analysis, unencrypted legacy protocols like **HTTP** and **FTP** offered zero data privacy. Because these protocols transmit data in clear, human-readable plain text, all data parameters were fully visible in the packet details and bytes panes.

For **HTTP traffic** generated by loading a local web server homepage, the browser's structural request methods (such as GET / HTTP/1.1) and the exact domain paths were completely exposed. Similarly, during an **FTP file transfer**, the security vulnerability was even more critical. User authentication commands (such as USER and PASS) were laid bare, revealing login credentials in plain text. Furthermore, the raw contents of the text files traveling across the network data stream were entirely readable, making them highly vulnerable to packet-sniffing interception.

#### 2. Encrypted Traffic Observations (HTTPS and SFTP)

In stark contrast, modern encrypted protocols like **HTTPS** and **SFTP** completely transformed the visibility of network packets by using robust cryptographic wrappers.

When loading a secure public website like Flipkart, standard **HTTPS traffic** hid all underlying application-layer details by passing data through a **TLS (Transport Layer Security)** tunnel. The

readable text fields were completely replaced by a secure block labeled "**Encrypted Application Data.**" Similarly, for **SFTP file transfers**, all client-server interactions were wrapped inside the secure **SSH (Secure Shell) protocol** on Port 22. In both cases, login credentials, web cookies, and transferred file payloads were scrambled into unreadable blocks of hexadecimal ciphertext.